DOSSIER PROJET

Nom de naissance : LEITE

Prénom : Stéphane

Adresse : 8 rue de Gomel, 63100 Clermont-Ferrand

Titre professionnel visé : Développeur web et web mobile — niveau 5

Modalité d'accès :

- ☑ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Organisme formateur : IPME

Sommaire

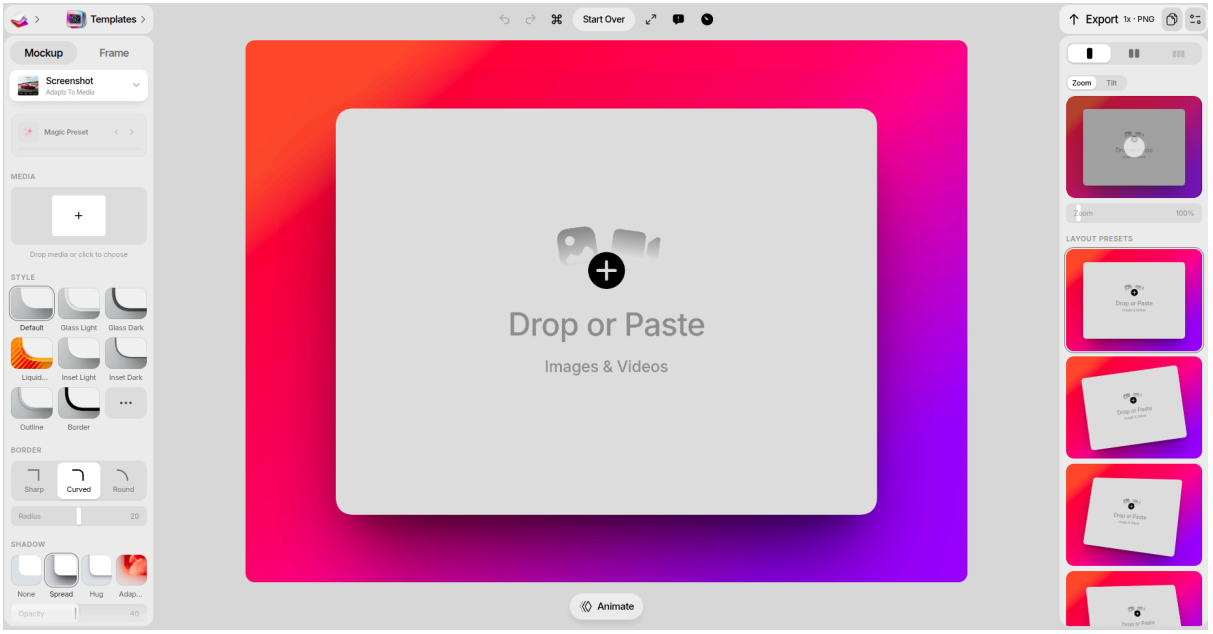
Sommaire....1
I — Introduction....2
A — Contexte du projet....2
B — Public cible....3
C — Objectifs globaux....4
II — Conception....4
A — UML....4
1 — Diagramme de cas d'utilisation....4
2 — Diagramme de classes....5
B — Charte graphique....9
1 — Couleurs....9
2 — Polices....10
3 — Logo....10
C — Maquettes....11
III — Outils et technologies....14
A — Outils....14
1 — Organisation....14
2 — Gestion de versions....14
B — Base de données....14
C — Technologies et langages....14
D — Dépendances....16
E — APIs....16
F — Outil de build....16
IV — Backend....17
A — Base de données....17
B — Les contrôleurs....18
1 — L'authentification....18

2 — Un exemple de contrôleur : PresentationController....20
3 — Méthode index (app_presentation_index)....20
4 — Méthode new (app_presentation_new)....21
5 — Méthode edit (app_presentation_edit)....21
6 — Méthode delete....22
7 — Service FileUploader....22
V — Frontend....23
A — Accessibilité, SEO et performance....23
1 — Accessibilité....23
2 — SEO....24
3 — Performance....24
B — CSS....26
C — JavaScript....27
1 — Vision globale....27
2 — presentation_editor_controller.js en détail....28
VI — Sécurité....31
VII — Tests....33
VIII — Déploiement....34
IX — Améliorations et perspectives d'évolution....36
A — Améliorations (proches du code actuel)....36
1 — Confirmation d'email et mot de passe oublié....36
2 — Durcissement des mots de passe....36
3 — Suppression du compte (RGPD)....36
4 — Drag & drop sur le canvas....36
5 — Miniature du dashboard....37
B — Perspectives d'évolution....37
1 — Galerie / partage public....37
2 — Templates de fonds....37
3 — Plusieurs calques / mockups device....37
X — Conclusion....38

I — Introduction

A — Contexte du projet

Briefed est un outil web d'édition de présentations visuelles, inspiré de services comme shots.so. L'utilisateur charge une image (capture d'écran, maquette, visuel produit), la place dans un cadre, choisit un fond (couleur unie, dégradé ou image), ajuste la position, l'échelle, la bordure, les coins arrondis et l'ombre, puis exporte le résultat en PNG.



shots.so

L'idée de Briefed m'est venue du constat suivant : quand on veut partager un design ou une capture d'écran (portfolio, réseaux, documentation, brief client), l'image brute est souvent trop « plate ». Il faut un fond, un format propre, une ombre, un cadre. Les outils du marché le font bien, mais je voulais comprendre comment reconstruire ce type d'éditeur de A à Z, avec un vrai backend (comptes, sauvegarde, réédition) et pas seulement un gadget côté navigateur.

Principe d'architecture central : on ne stocke jamais l'image aplatie / exportée. On stocke une « recette » : le fichier source, le fond, le format, la position (posX / posY), l'échelle (scale), la bordure, l'ombre, etc. Le canvas rejoue cette recette à chaque ouverture. C'est ce qui permet de rééditer une présentation plus tard sans perdre la qualité ni les réglages.

J'ai choisi ce projet pour faire le pont entre toutes les disciplines dont je suis issu. Avant de vouloir devenir développeur, je travaillais dans le marketing et le design, je connais donc le panorama des différents outils de design disponibles sur le marché. Suite à mon premier master en marketing, j'ai eu une période de transition entrepreneuriale où j'ai eu l'idée de créer des outils et applications, sans aucun background de code à ce moment. C'est là que j'ai eu l'idée de Briefed, initialement c'était un outil de générateur de briefs créatifs pour les juniors et personnes en reconversion dans les métiers du design, qui manquent souvent de projets professionnels dans leur portfolio. L'ambition finale est de créer une suite d'outils utilitaires pour créatif, que ce soit les designers, les marketeurs, les développeurs front, et l'outil d'édition de présentation est le premier de la liste.

B — Public cible

Le public cible est constitué de personnes qui ont besoin de valoriser des visuels :

- designers et développeurs (captures de maquettes ou d'interfaces) ;
- indépendants et petites équipes produit (visuels pour un brief, un portfolio, LinkedIn, Dribbble) ;

- formateurs ou étudiants qui veulent présenter un travail proprement.

Ce public n'a pas forcément besoin d'un outil de diaporama type PowerPoint : il a besoin d'un **éditeur simple, rapide, avec un rendu soigné**, et de pouvoir **retrouver et modifier** ses créations plus tard.

C — Objectifs globaux

J'avais plusieurs idées (galerie publique, templates, drag & drop avancé, comptes admin…). Pour rester réaliste dans le délai, j'ai défini un Produit Minimum Viable (MVP) :

- **Inscription / connexion / déconnexion** : un utilisateur peut créer un compte et accéder à son espace.
- **Liste des présentations** (« Dashboard ») : uniquement les présentations de l'utilisateur connecté.
- **Création et édition** dans un éditeur unique (sidebar + canvas), façon shots.so : on crée une présentation avec des valeurs par défaut, puis on ajuste le visuel.
- **Upload d'image** (visuel principal et fond image) avec validation (JPEG, PNG, WebP, 2 Mo max).
- **Réglages visuels persistés** : format (carré 1:1, paysage 16:9, standard 4:3), fond (solide / dégradé / image), position, échelle, bordure, radius, ombre.
- **Export PNG** côté navigateur (le PNG n'est pas stocké en base).
- **Suppression** d'une présentation, avec jeton CSRF et suppression du fichier uploadé.
- **Contrôle de propriété** : un utilisateur ne peut ni voir, ni modifier, ni supprimer la présentation d'un autre (protection IDOR).

II — Conception

A — UML

Pour réaliser mes diagrammes UML, j'ai utilisé draw.io pour le cas d'utilisation et pour le diagramme de classes.

1 — Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les types d'utilisateurs, ce qu'ils peuvent faire, et les fonctionnalités principales.

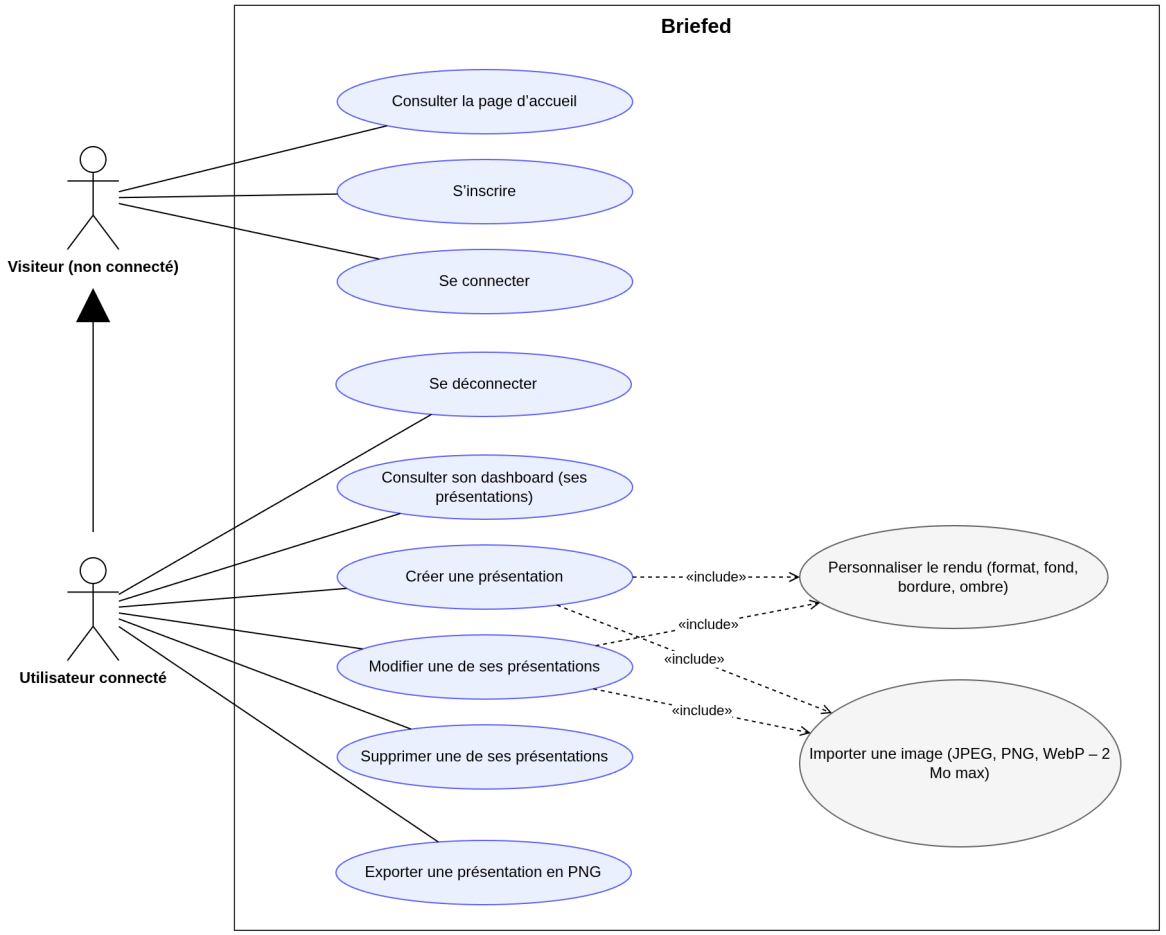
J'ai identifié **2 acteurs** (pas de rôle administrateur dans cette version) :

L'utilisateur non connecté peut :

- consulter la page d'accueil ;
- s'inscrire ;
- se connecter.

L'utilisateur connecté peut :

- se déconnecter ;
- consulter son dashboard (liste de ses présentations) ;
- créer une présentation ;
- modifier une de ses présentations (éditeur canvas) ;
- exporter une présentation en PNG ;
- supprimer une de ses présentations.



2 — Diagramme de classes

Le diagramme de classes décrit la structure du code et de la base : entités, attributs, relations.

J'ai identifié **trois classes** :

- **User** : un utilisateur, avec un id, un email unique, un password (haché), des roles. Un utilisateur possède plusieurs présentations.

```php
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 180)]
    #[Assert\NotBlank(message: 'L\'email est obligatoire.')]
    #[Assert\Email(message: 'Cet email n\'est pas valide.')]
    private ?string $email = null;

    /**
     * @var list<string> The user roles
     */
    #[ORM\Column]
    private array $roles = [];

    /**
     * @var string The hashed password
     */
    #[ORM\Column]
    private ?string $password = null;

    /**
     * @var Collection<int, Presentation>
     */
    #[ORM\OneToMany(targetEntity: Presentation::class, mappedBy: 'owner')]
    private Collection $presentations;
```

- **Presentation** : une présentation, avec un titre, le nom du fichier image source (imageFilename), un format, des coordonnées (posX, posY), une échelle (scale), une bordure (border, borderColor, borderOpacity), un radius, une ombre (shadowType, shadowOpacity, shadowAngle), des dates createdAt / updatedAt.

```php
#[ORM\Entity(repositoryClass: PresentationRepository::class)]
class Presentation
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\ManyToOne(inversedBy: 'presentations')]
    #[ORM\JoinColumn(nullable: false)]
    private ?User $owner = null;

    #[ORM\Column(length: 255)]
    #[Assert\NotBlank(message: 'Le titre est obligatoire.')]
    #[Assert\Length(max: 255, maxMessage: 'Le titre ne peut pas dépasser {{ limit }} caractères.')]
    private ?string $title = null;

    #[ORM\Column(length: 255, nullable: true)]
    private ?string $imageFilename = null;

    #[ORM\ManyToOne(inversedBy: 'presentations')]
    #[ORM\JoinColumn(nullable: false)]
    private ?Background $background = null;

    #[ORM\Column(length: 20)]
    #[Assert\Choice(choices: ['carre', 'paysage', 'standard'], message: 'Format non supporté.')]
    private ?string $format = null;

    #[ORM\Column]
    private ?float $posX = null;

    #[ORM\Column]
    private ?float $posY = null;

    #[ORM\Column]
    #[Assert\Range(min: 0.1, max: 5.0, notInRangeMessage: 'L\'échelle doit être entre {{ min }} et
    private ?float $scale = null;
```

- **Background** : le fond d'une présentation. Type (solid, degrade, image), color1, color2 (optionnelle), gradientAngle, imageFilename (si fond image).

```php
class Background
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    /**
     * @var Collection<int, Presentation>
     */
    #[ORM\OneToMany(targetEntity: Presentation::class, mappedBy: 'background')]
    private Collection $presentations;

    #[ORM\Column(length: 20, nullable: false)]
    #[Assert\NotBlank]
    #[Assert\Choice(choices: ['solid', 'degrade', 'image'], message: 'Type de fond non supporté.')]
    private ?string $type = null;

    #[ORM\Column(length: 7, nullable: false)]
    #[Assert\NotBlank]
    #[Assert\Regex(
        pattern: '/^#[0-9a-fA-F]{6}$/',
        message: 'La couleur doit être au format hexadécimal (ex: #1a1a1a).'
    )]
    private ?string $color1 = null;

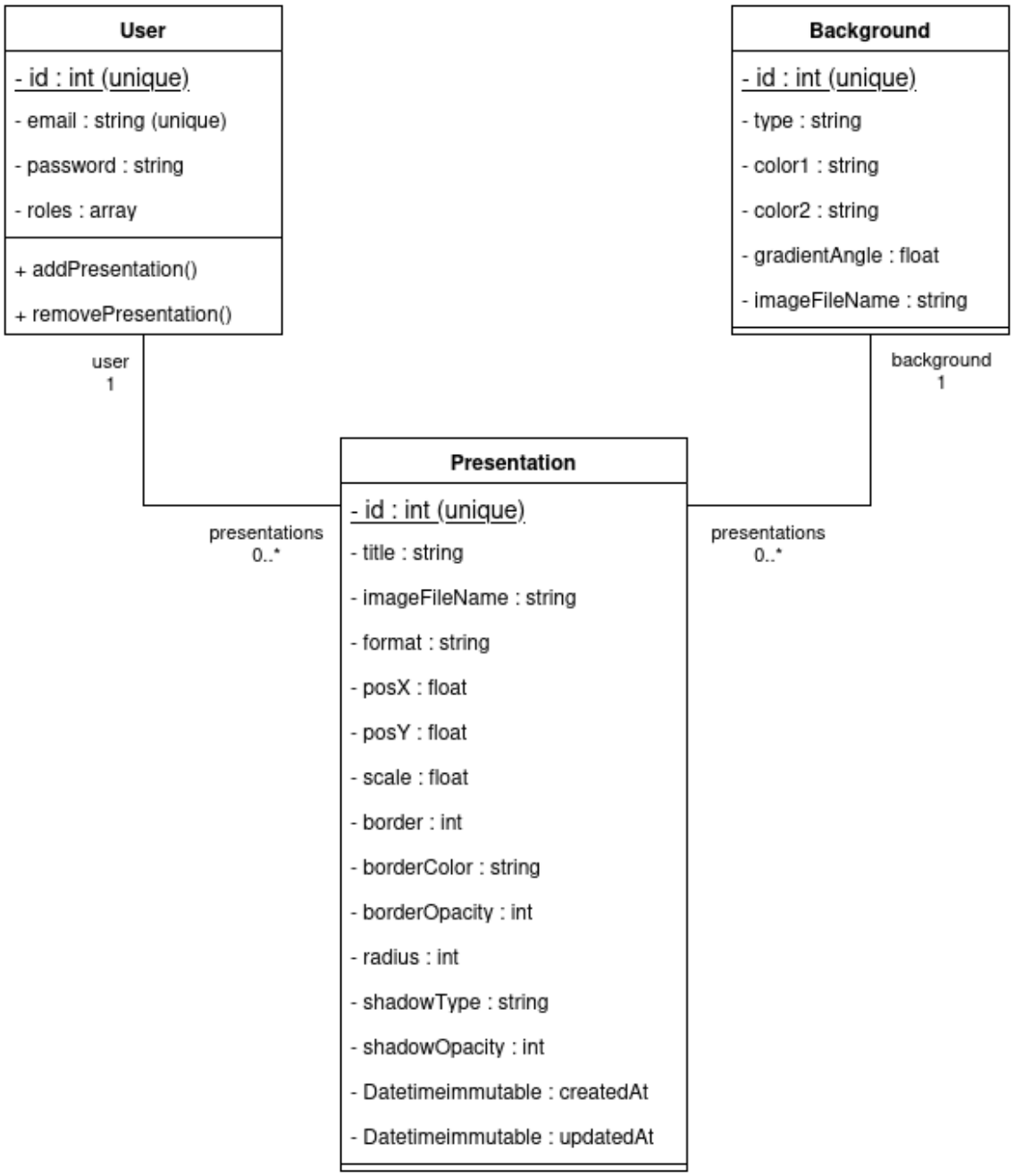
    #[ORM\Column(length: 7, nullable: true)]
    #[Assert\Regex(
        pattern: '/^#[0-9a-fA-F]{6}$/',
        message: 'La couleur doit être au format hexadécimal (ex: #1a1a1a).'
    )]
    private ?string $color2 = null;

    #[ORM\Column(nullable: true)]
    #[Assert\Range(min: 0, max: 360, notInRangeMessage: 'L\'angle du dégradé doit être entre {{ mi
    private ?float $gradientAngle = null;

    #[ORM\Column(length: 255, nullable: true)]
    private ?string $imageFilename = null;
```

Relations :

- Un **User** peut avoir **plusieurs Presentation** ; une Presentation a **un seul** owner (ManyToOne / OneToMany).
- Une **Presentation** a **un Background** obligatoire ; un Background peut être lié à plusieurs présentations (ManyToOne / OneToMany).



B — Charte graphique

J'ai défini une charte pour une identité claire, moderne, « produit SaaS », avec un contraste suffisant pour la lisibilité.

1 — Couleurs

Les couleurs sont centralisées dans des **variables CSS** (assets/styles/_tokens.css) :

Token	Valeur	Usage
`--color-primary`	`#6366f1`	boutons principaux, accents, logo
`--color-gradient-start`	`#a78bfa`	dégradés d'ambiance
`--color-gradient-end`	`#6366f1`	dégradés d'ambiance
`--color-bg`	`#ffffff`	fond des pages
`--color-bg-app`	`#e1e2e4`	fond autour du canvas
`--color-bg-canvas`	`#f9f9f9`	zone de travail
`--color-bg-input`	`#f3f4f6`	champs
`--color-bg-hero`	`#928eb6`	section d'accueil
`--color-text`	`#111827`	titres
`--color-text-body`	`#4b5563`	paragraphes
`--color-text-muted`	`#6b7280`	textes secondaires
`--color-text-inverse`	`#ffffff`	texte sur fond coloré
`--color-border`	`#e5e7eb`	bordures

Cette palette indigo / violet sur fond clair évoque un outil créatif sans être trop saturée. Les boutons primaires contrastent avec le fond blanc.

2 — Polices

Police de base : Poppins (Google Fonts), sans-serif.

Utilisée pour toute l'interface (titres, boutons, labels, paragraphes). Elle est moderne, lisible, adaptée à un produit web. Un seul fichier de police simplifie la charge et garde une identité cohérente.

3 — Logo

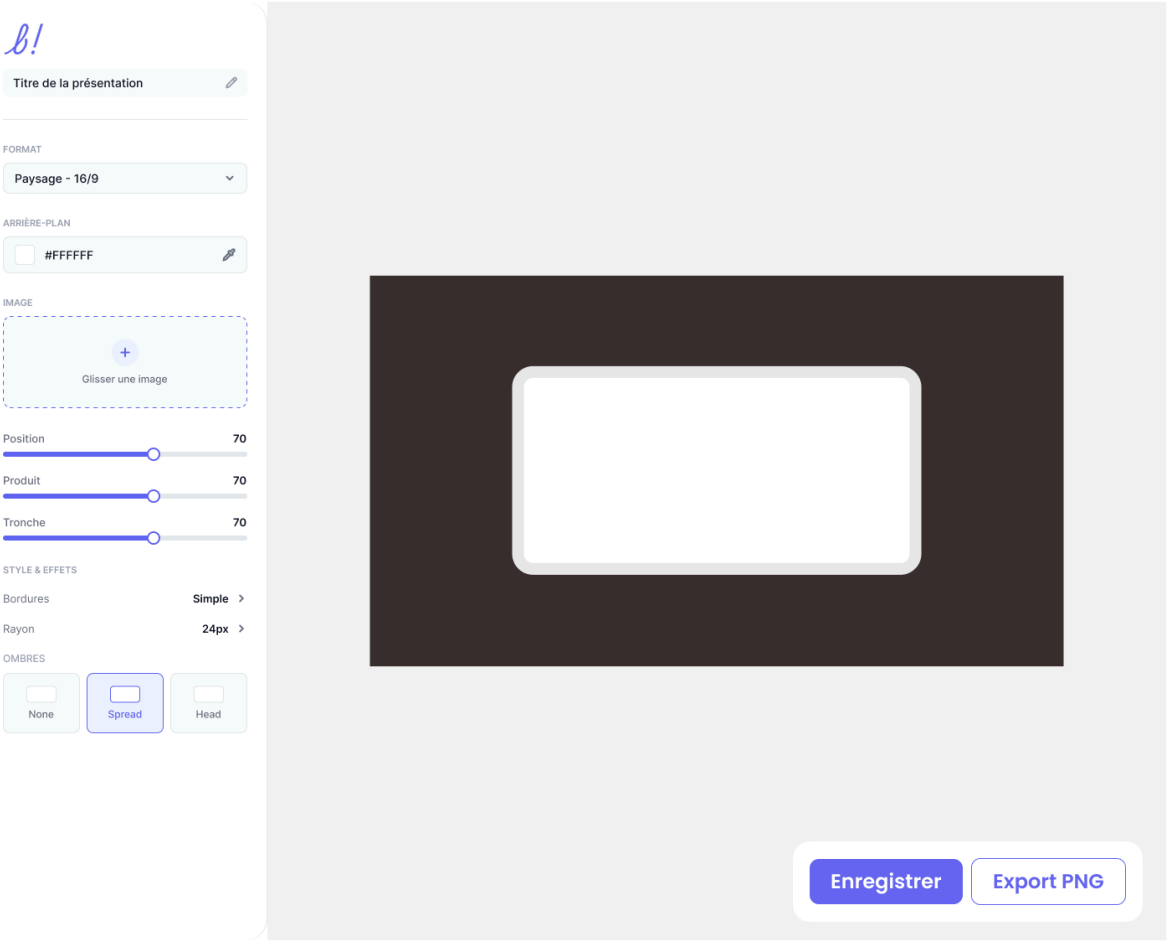
Le logo est le mot **briefed!** en script minuscule, couleur primaire (#6366f1), avec un point d'exclamation. Une version icône (b!) est utilisée dans le header de l'éditeur.

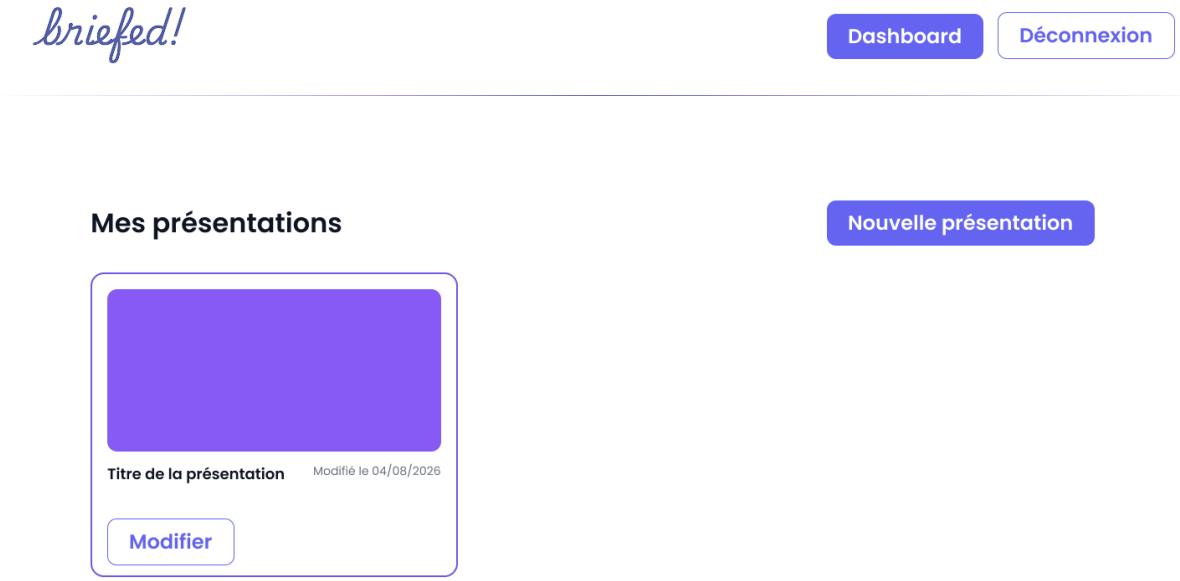


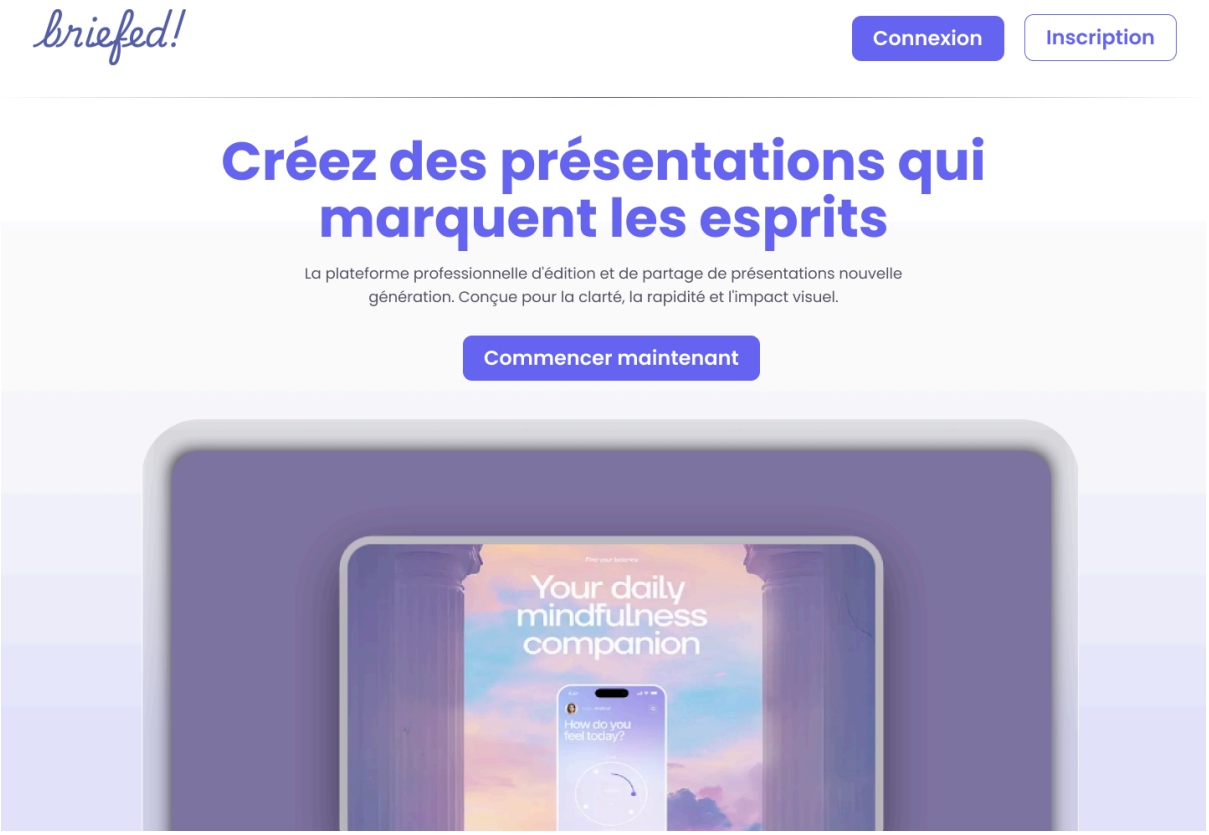
C — Maquettes

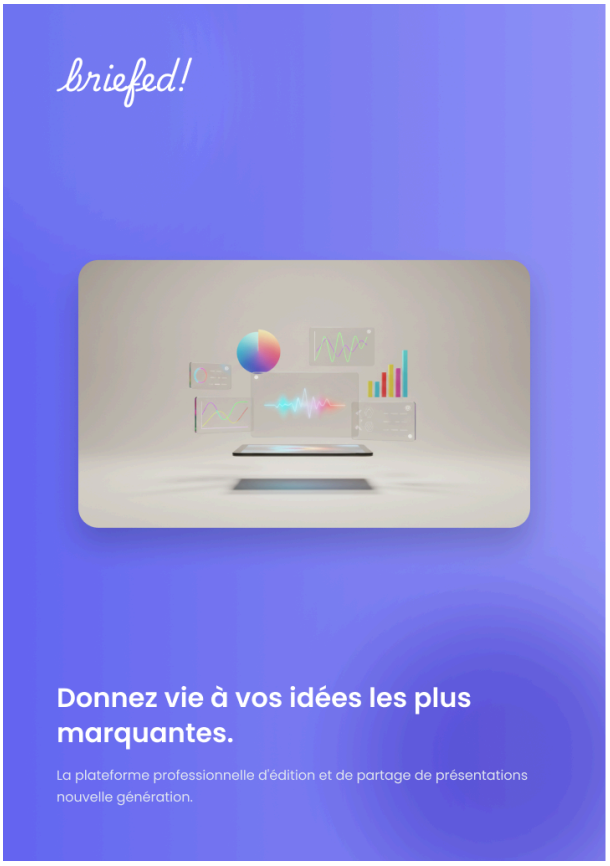
Pages prévues :

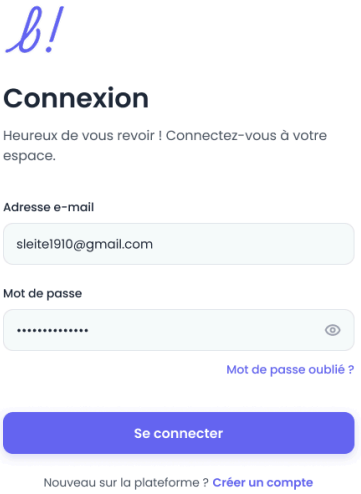
- accueil ;
- inscription / connexion ;
- dashboard (liste des présentations) ;
- éditeur (sidebar + canvas).

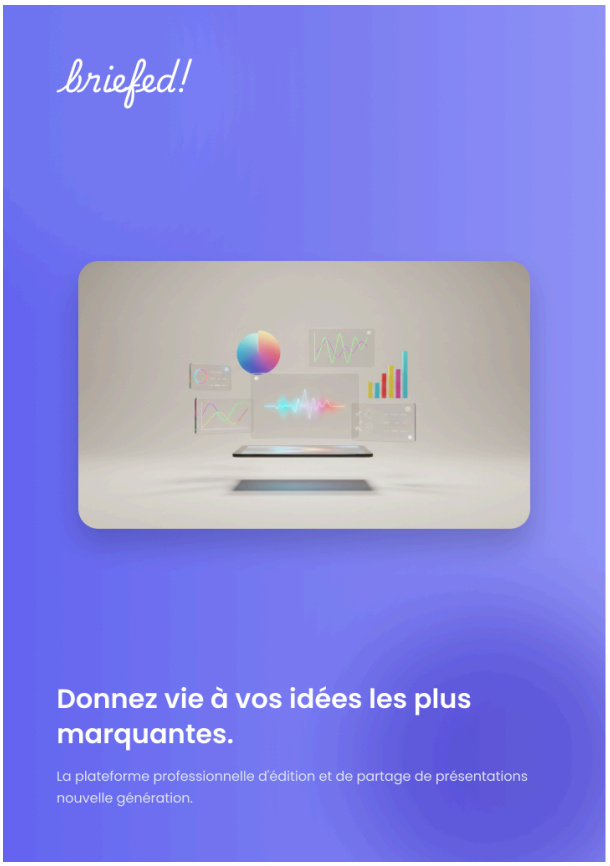


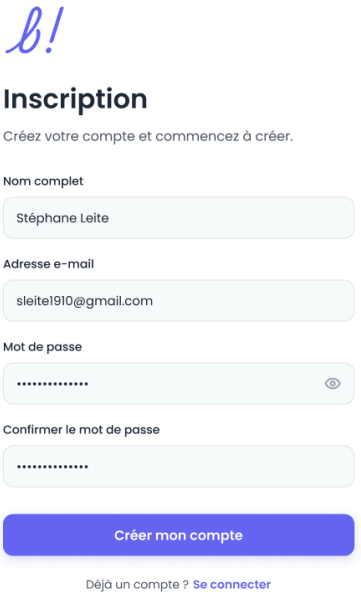

briefed!
Dashboard
Déconnexion
Mes présentations
Nouvelle présentation
Titre de la présentation
Modifié le 04/08/2026
Modifier


briefed!
Connexion
Inscription
Créez des présentations qui marquent les esprits
La plateforme professionnelle d'édition et de partage de présentations nouvelle génération. Conçue pour la clarté, la rapidité et l'impact visuel.
Commencer maintenant
Your daily mindfulness companion
How do you feel today?


briefed!
Donnez vie à vos idées les plus marquantes.
La plateforme professionnelle d'édition et de partage de présentations nouvelle génération.


Connexion
Heureux de vous revoir ! Connectez-vous à votre espace.
Adresse e-mail
sleite1910@gmail.com
Mot de passe
Mot de passe oublié ?
Se connecter
Nouveau sur la plateforme ? Créer un compte


briefed!
Donnez vie à vos idées les plus marquantes.
La plateforme professionnelle d'édition et de partage de présentations nouvelle génération.


Inscription
Créez votre compte et commencez à créer.
Nom complet
Stéphane Leite
Adresse e-mail
sleite1910@gmail.com
Mot de passe
Confirmer le mot de passe
Créer mon compte
Déjà un compte ? Se connecter

III — Outils et technologies

A — Outils

1 — Organisation

Pour m'organiser, j'ai utilisé Notion pour écrire mes tâches et suivre l'avancée de mon projet.

2 — Gestion de versions

J'ai utilisé **Git** pour le suivi des versions et **GitHub** pour le dépôt distant.

Commandes principalement utilisées :

1. git add — ajouter des fichiers au prochain commit
2. git commit -m "…" — enregistrer un état
3. git push — envoyer vers GitHub
4. git pull — récupérer les commits distants

Pour accéder à mon repo GitHub : https://github.com/slt-631/briefed-v2-cda

B — Base de données

J'ai utilisé **MariaDB** (SGBDR) pour stocker les utilisateurs, les présentations et les fonds. En local, MariaDB tourne via Docker Compose (compose.yaml), avec phpMyAdmin pour inspecter les tables.

MariaDB est compatible MySQL, fiable, et largement utilisé en développement web.

On ne stocke **que les métadonnées** et le **nom de fichier** des images. Les fichiers eux-mêmes sont dans public/uploads/ (hors Git).

C — Technologies et langages

Après la conception (UML, maquettes), j'ai structuré l'application avec **Symfony 7.4**, un framework PHP.

J'ai choisi Symfony parce que nous l'avons utilisé en formation, qu'il est mature, bien documenté, et qu'il impose une architecture **MVC** (Model-View-Controller) : séparation entre données (entités Doctrine), logique (contrôleurs / services) et affichage (Twig).

Doctrine ORM fait le lien entre les classes PHP et les tables SQL. Je travaille avec des objets ; Doctrine génère des requêtes préparées, ce qui limite les injections SQL.

Twig est le moteur de templates : HTML, boucles, conditions, héritage de layouts (base.html.twig), sans mélanger le PHP métier dans les vues.

Côté front :

- HTML / CSS (fichiers modulaires) ;
- **JavaScript** via un contrôleur **Stimulus** (presentation_editor_controller.js) ;
- **API Canvas** du navigateur pour dessiner le rendu et l'exporter en PNG.

Pas de React / Vue : le besoin est un éditeur ciblé, pas une SPA complète.

Pourquoi ces choix plutôt que d'autres ?

- **PHP 8.2+** : le projet s'appuie sur les nouveautés récentes du langage, notamment le typage des propriétés et les **attributs** natifs. Ils sont utilisés partout dans Briefed : #[Route] pour les routes, #[IsGranted] pour la sécurité, #[Assert\…] pour la validation, #[ORM\…] pour le mapping Doctrine. La route, la règle de sécurité ou la contrainte est écrite juste au-dessus de la méthode ou de la propriété concernée, ce qui rend le code plus lisible qu'une configuration séparée en YAML.
- **Symfony 7.4** : c'est une version **LTS** (Long Term Support), maintenue et corrigée pendant plusieurs années : un choix rassurant pour une application qu'on voudrait faire vivre. Symfony fournit aussi le **Maker Bundle** (make:entity, make:user, make:auth, make:registration-form), qui génère le code de base et m'a fait gagner du temps sur les parties répétitives.
- **Doctrine ORM + Doctrine Migrations** : chaque modification du schéma de la base génère un fichier de migration versionné dans le dossier migrations/. La base peut ainsi être recréée à l'identique sur n'importe quelle machine, ce qui évite les problèmes du type « chez moi ça marche ».
- **Symfony Form et Validator** : les formulaires (PresentationType, BackgroundType, RegistrationFormType) sont décrits en PHP, générés dans Twig et validés côté serveur, avec des messages d'erreur en français.
- **Stimulus et Turbo** (Symfony UX) : Stimulus ajoute de l'interactivité à du HTML rendu par Twig (voir partie V.C), et Turbo rend la navigation plus fluide sans recharger toute la page, sans que j'aie à écrire de JavaScript pour cela.
- **API Canvas 2D** : native dans tous les navigateurs modernes. J'aurais pu utiliser une bibliothèque de dessin (Fabric.js, Konva…), mais l'API native suffit pour ce besoin : elle n'ajoute aucune dépendance et m'a obligé à comprendre chaque étape du rendu.
- **Docker Compose** : la base MariaDB 10.11 et phpMyAdmin tournent dans des conteneurs définis dans compose.yaml. L'environnement se lance en une commande (docker compose up -d), toujours dans la même version, et les ports utilisés (3308 pour la base, 8080 pour phpMyAdmin) évitent les conflits avec d'autres installations locales.

Récapitulatif de la stack :

Couche	**Technologie**	**Rôle dans Briefed**
Langage serveur	PHP 8.2+	logique métier
Framework	Symfony 7.4 (LTS)	routing, sécurité, formulaires, architecture MVC
ORM	Doctrine ORM + Migrations	lien entités ↔ tables, historique du schéma

Couche	Technologie	Rôle dans Briefed
Base de données	MariaDB 10.11 (Docker)	utilisateurs, présentations, fonds
Templates	Twig	génération des pages HTML
Front	HTML5, CSS3 (variables CSS)	structure et mise en forme
JavaScript	Stimulus 3 + Turbo 8	éditeur interactif, navigation fluide
Rendu graphique	API Canvas 2D	aperçu en direct et export PNG
Assets	Asset Mapper + importmap	chargement du JS et du CSS sans build
Outils	Git / GitHub, Docker, phpMyAdmin, Notion	versions, environnement, organisation

D — Dépendances

Dépendances principales (Composer) :

- symfony/framework-bundle, security-bundle, form, validator, twig-bundle
- doctrine/orm + migrations
- symfony/stimulus-bundle et symfony/ux-turbo (Stimulus / Turbo)
- symfony/asset-mapper pour les assets

Côté navigateur (Importmap, pas npm/Webpack) :

- @hotwired/stimulus
- @hotwired/turbo
- Font Awesome (CDN) pour les icônes
- Google Fonts (Poppins)

E — APIs

Aucune API externe métier (pas de carte, pas de géocodage). L'export PNG utilise l'API Canvas du navigateur (`canvas.toDataURL("image/png")`).

F — Outil de build

J'utilise **Asset Mapper** (intégré à Symfony 7), pas Webpack Encore.

Asset Mapper sert les fichiers JS/CSS du dossier assets/ via une importmap. Moins d'outillage Node à maintenir, suffisant pour ce projet (un contrôleur Stimulus + du CSS).

IV — Backend

A — Base de données

Pour structurer la base, j'ai créé les entités avec symfony console make:entity (et make:user pour User).

Chaque commande génère :

- l'**entité** (classe = table) ;
- le **repository** (requêtes find, findAll, findBy… et requêtes custom).

Entités :

1. **User** (make:user) — email unique, mot de passe, rôles, collection de présentations.
2. **Presentation** — recette visuelle + owner + background.
3. **Background** — type de fond et couleurs / image associés.

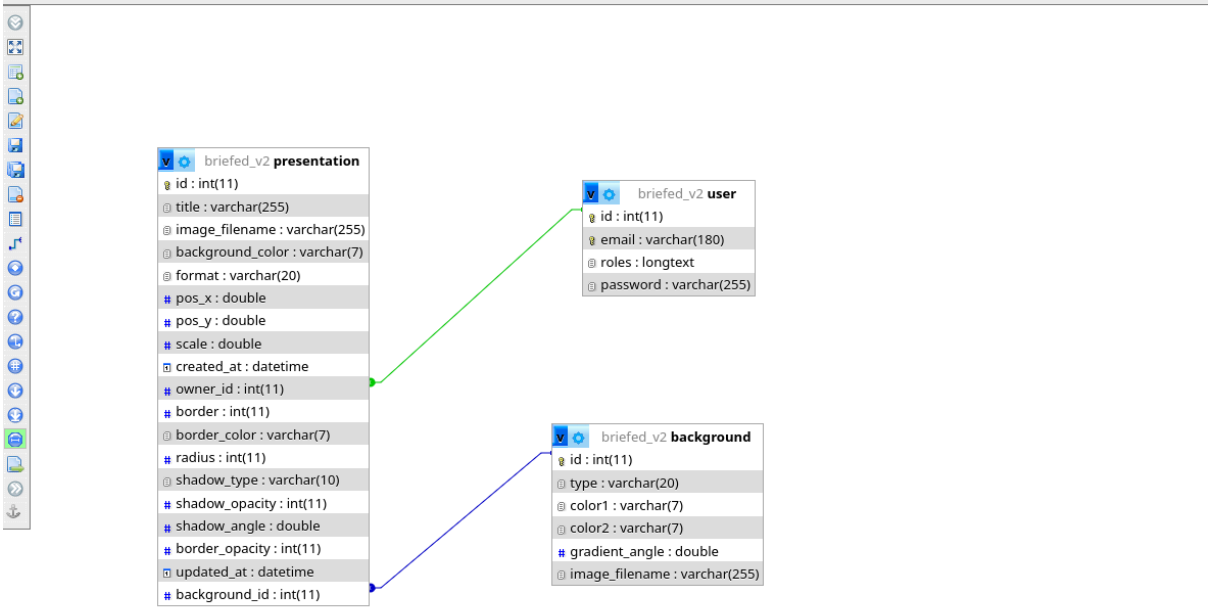
Valeurs par défaut dans le constructeur de Presentation (titre « Ma presentation », format paysage, échelle 1, bordure 0, ombre none, etc.) : à la création, l'éditeur est déjà utilisable.

Ensuite : symfony console make:migration puis doctrine:migrations:migrate. Doctrine transforme les classes en tables SQL et garde un historique des évolutions (table doctrine_migration_versions).

Méthode custom dans PresentationRepository :

```
public function findByOwner(User $owner): array
{
   return $this->createQueryBuilder('p')
      ->where('p.owner = :owner')
      ->setParameter('owner', $owner)
      ->orderBy('p.createdAt', 'DESC')
      ->getQuery()
      ->getResult();
}
```

Le dashboard n'affiche **jamais** toutes les présentations de la base, seulement celles de l'utilisateur connecté, via un paramètre lié (requête préparée).



B — Les contrôleurs

Les contrôleurs reçoivent les requêtes HTTP, appellent Doctrine / services, et rendent une vue Twig (ou redirigent). Les **routes** (`#[Route]`) lient une URL à une méthode.

Contrôleur	**Rôle**
HomeController	page d'accueil /
RegistrationController	inscription /register
SecurityController	connexion /login, déconnexion /logout
PresentationController	CRUD des présentations sous /presentations

Contrôleurs du projet :PresentationController est protégé par #[IsGranted('ROLE_USER')] : inaccessible sans compte.

1 — L'authentification

J'ai généré l'auth avec make:auth et make:registration-form.

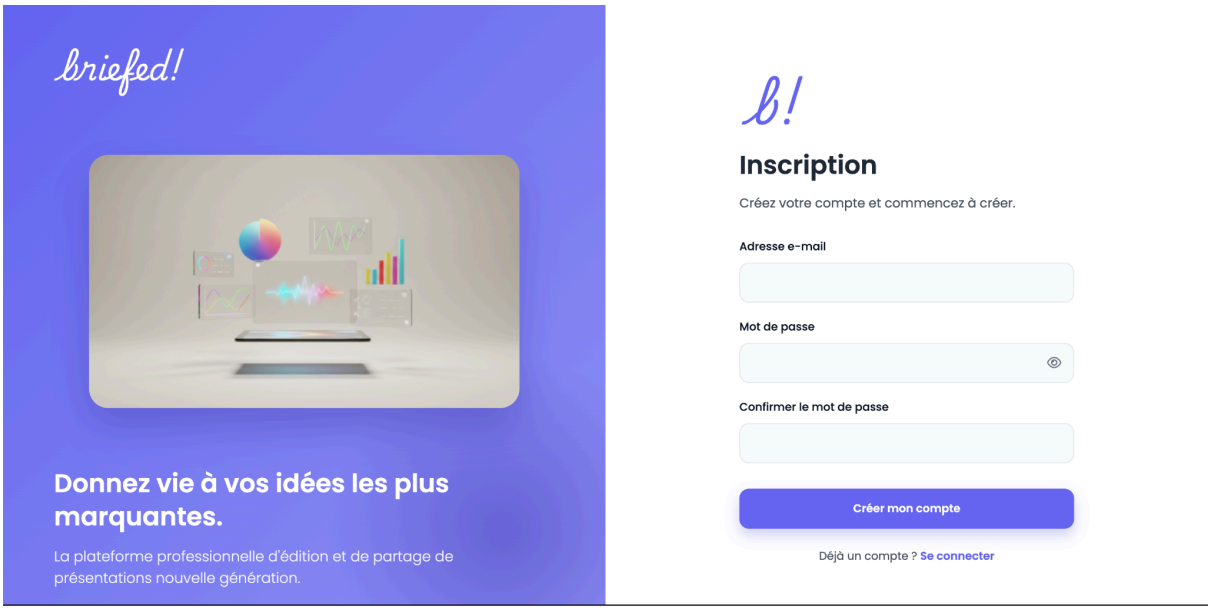
À l'inscription (RegistrationController) :

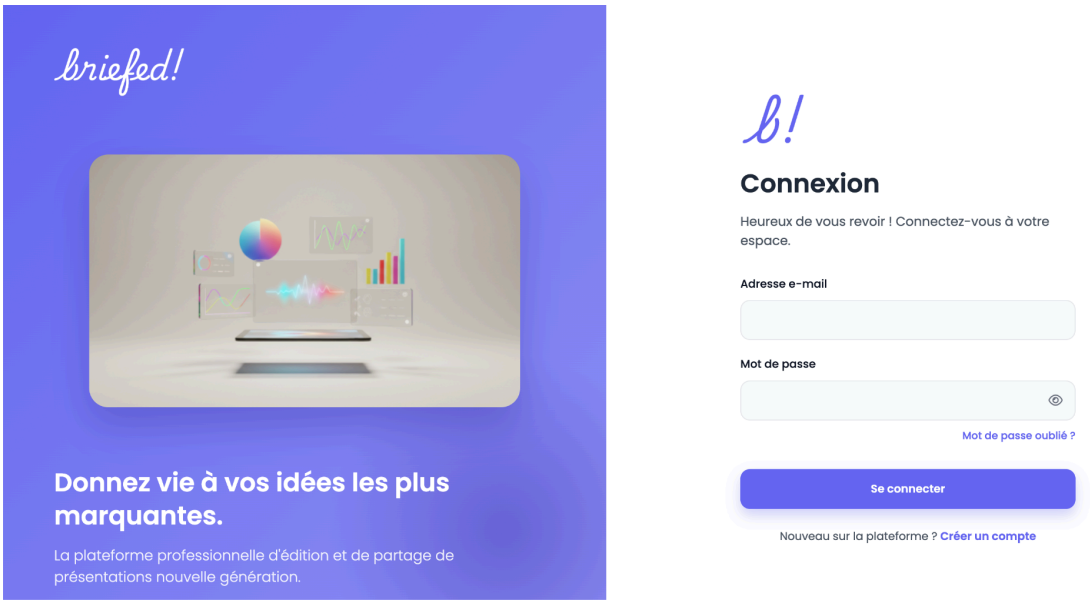
1. formulaire RegistrationFormType (email + mot de passe en clair, champ plainPassword **non mappé** sur l'entité) ;
2. si valide, le mot de passe est haché avec UserPasswordHasherInterface ;
3. persist + flush ;
4. redirection vers la page de connexion.

Contraintes actuelles du mot de passe : obligatoire, **minimum 6 caractères**.

Le hasher Symfony est en `auto` (`security.yaml`) : algorithme fort (bcrypt / argon2 selon l'environnement). Le mot de passe n'est **jamais** stocké en clair.

La connexion utilise `form_login` avec **CSRF** (`enable_csrf: true`).





2 — Un exemple de contrôleur : `PresentationController`

Routes :

Route	Nom	Méthode	Action
`/presentations`	`app_presentation_index`	GET	liste
`/presentations/new`	`app_presentation_new`	GET/POST	création
`/presentations/{id}/edit`	`app_presentation_edit`	GET/POST	édition
`/presentations/{id}/delete`	`app_presentation_delete`	POST	suppression

```php
#[Route('/presentations')]
#[IsGranted('ROLE_USER')]
class PresentationController extends AbstractController
{
    #[Route('', name: 'app_presentation_index', methods: ['GET'])]
    public function index(PresentationRepository $repository): Response
    {
        /** @var User $user */
        $user = $this->getUser();

        $presentations = $repository->findByOwner($user);

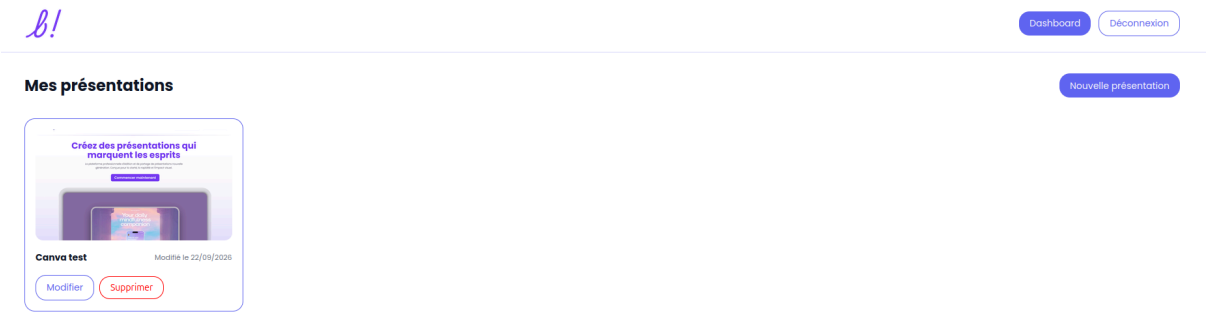
        return $this->render('presentation/index.html.twig', [
            'presentations' => $presentations,
        ]);


    }
```

3 — Méthode `index` (`app_presentation_index`)

1. Récupère l'utilisateur connecté ($this->getUser()).
2. Appelle findByOwner($user).
3. Passe la liste au template presentation/index.html.twig.

Le dashboard affiche titre, date de modification, miniature si une image existe, boutons Modifier / Supprimer.



4 — Méthode `new` (`app_presentation_new`)

1. Instancie une Presentation (valeurs par défaut du constructeur).
2. Assigne l'owner = utilisateur courant (pas via le formulaire : l'owner n'est pas un champ falsifiable).
3. Crée le formulaire PresentationType (qui embarque BackgroundType en sous-formulaire).
4. Si soumis et valide :
   - upload de l'image principale et éventuellement de l'image de fond (FileUploader) ;
   - persist du Background puis de la Presentation ;
   - flush ;
   - redirection vers le dashboard.
5. Sinon, affiche l'éditeur (presentation/new.html.twig → partial canvas.html.twig).

persist prépare l'INSERT ; flush l'exécute.

5 — Méthode `edit` (`app_presentation_edit`)

Symfony hydrate la Presentation via le {id} de l'URL (param converter).

1. **Contrôle de propriété** : si getOwner() !== getUser(), AccessDeniedException (403). Un utilisateur ne peut pas éditer l'id d'un autre en changeant l'URL.
2. Formulaire pré-rempli avec la recette en base.
3. Si valide : nouveaux uploads éventuels, setUpdatedAt(now), persist du background, flush, redirection.

En SQL, cela correspond à un UPDATE presentation SET … WHERE id = … (généré par Doctrine).

6 — Méthode `delete`

1. Même contrôle de propriété.
2. Vérification du jeton CSRF (delete + id).
3. remove + flush.
4. Suppression du fichier image via FileUploader::remove (la base ne garde pas le binaire).

```php
#[Route('/{id}/delete', name: 'app_presentation_delete', methods: ['POST'])]
public function delete(Presentation $presentation, Request $request, EntityManagerInterface $em, FileUploader $uploade
{
    if ($presentation->getOwner() !== $this->getUser()) {
        throw $this->createAccessDeniedException();
    }

    if ($this->isCsrfTokenValid('delete' . $presentation->getId(), $request->getPayload()->getString('_token'))) {
        $filename = $presentation->getImageFilename();

        $em->remove($presentation);
        $em->flush();

        if ($filename) {
            $uploader->remove($filename);
        }
    }
```

7 — Service `FileUploader`

Service dédié (injecté par autowiring), dossier public/uploads :

- **upload** : slugifie le nom original + uniqid() + extension réelle (guessExtension()), pour éviter les collisions et les noms dangereux ;
- **remove** : unlink si le fichier existe.

Le formulaire valide aussi le type MIME réel et la taille (Image constraint : jpeg/png/webp, 2 Mo).

```php
class FileUploader
{
    public function __construct(
        private string $uploadsDirectory,
        private SluggerInterface $slugger,
    ) {
    }

    public function upload(UploadedFile $file): string
    {
        $originalName = pathinfo($file->getClientOriginalName(), PATHINFO_FILENAME);
        $safeName = $this->slugger->slug($originalName)->lower();
        $newFilename = $safeName . '-' . uniqid() . '.' . $file->guessExtension();

        $file->move($this->uploadsDirectory, $newFilename);

        return $newFilename;
    }

    public function remove(string $filename): void
    {
        $path = $this->uploadsDirectory . '/' . $filename;

        if (is_file($path)) {
            unlink($path);
        }
    }
}
```

V — Frontend

Le front rend l'éditeur utilisable : sidebar de réglages + aperçu canvas, interface responsive, templates Twig réutilisables.

base.html.twig pose le HTML commun (charset, titre, Poppins, Font Awesome, importmap Asset Mapper). Les pages injectent leur contenu dans {% block body %}. Le header (partial/header.html.twig) change selon que l'utilisateur est connecté (Dashboard / Déconnexion) ou non (Connexion / Inscription).

L'éditeur (création et modification) est un **seul partial** templates/partial/canvas.html.twig : même UI, label de bouton différent (« Créer » / « Enregistrer »).

A — Accessibilité, SEO et performance

1 — Accessibilité

- attributs alt sur les images importantes (logo, choix de format / fond / ombre, aperçus) ;
- labels de formulaires Symfony ;
- contraste texte / fond via la charte ;
- police Poppins lisible.

2 — SEO

Briefed est surtout utilisé une fois connecté : le dashboard et l'éditeur sont des pages privées, qui n'ont pas vocation à apparaître dans Google. Le référencement concerne donc surtout la **page d'accueil**, qui présente le produit, et les pages publiques de **connexion** et d'**inscription**.

Ce qui est en place :

- **Un titre par page** : chaque template remplit le {% block title %} hérité de base.html.twig. L'accueil affiche « briefed! — Présentations nouvelle génération », le dashboard « Mes présentations », la création « Nouvelle présentation », et en édition le titre de la présentation elle-même. La balise <title> est l'un des éléments les plus pris en compte par les moteurs de recherche. C'est aussi le texte affiché dans l'onglet du navigateur et dans les favoris.
- **Des balises sémantiques** : <header> pour l'en-tête du site (partial/header.html.twig), <nav> pour les liens de navigation, <main> et <section> pour le contenu de la page d'accueil. Elles décrivent le rôle de chaque zone, ce qui aide les robots d'indexation (et les lecteurs d'écran) à comprendre la page, là où une suite de n'apporte aucune information.
- **Une hiérarchie de titres cohérente** : un seul <h1> par page (« Créez des présentations qui marquent les esprits » sur l'accueil, « Mes présentations » sur le dashboard), puis des <h2> pour le titre de chaque carte de présentation.
- **Des URLs lisibles** : grâce aux attributs #[Route], les adresses sont claires et parlantes (/presentations, /presentations/new, /presentations/{id}/edit, /login, /register) plutôt que des paramètres techniques du type index.php?page=3.
- **Des textes alternatifs** sur les images : alt="briefed!" sur le logo, alt égal au titre de la présentation sur les miniatures du dashboard. Ils servent au référencement des images autant qu'à l'accessibilité.
- **Un site rapide** (voir partie suivante) : la vitesse de chargement fait partie des critères de classement de Google (Core Web Vitals).
- **Les balises techniques** du head : la langue de la page (html lang="fr"), la balise viewport qui adapte l'affichage à la largeur réelle de l'écran, et une meta description sur la page d'accueil. Ces trois éléments sont contrôlés par Lighthouse.

Ce qu'il faudrait ajouter pour aller plus loin :

- une balise <meta name="robots" content="noindex"> sur les pages privées (dashboard, éditeur) pour qu'elles ne soient jamais indexées ;
- des balises **Open Graph** (og:title, og:description, og:image) pour afficher un bel aperçu quand le lien du site est partagé sur un réseau social ;
- un fichier sitemap.xml et un robots.txt pour guider les robots vers les pages publiques.

3 — Performance

Même si Briefed est une petite application, j'ai fait attention à ce que les pages se chargent vite et à ce que l'éditeur reste fluide.

Côté chargement des pages :

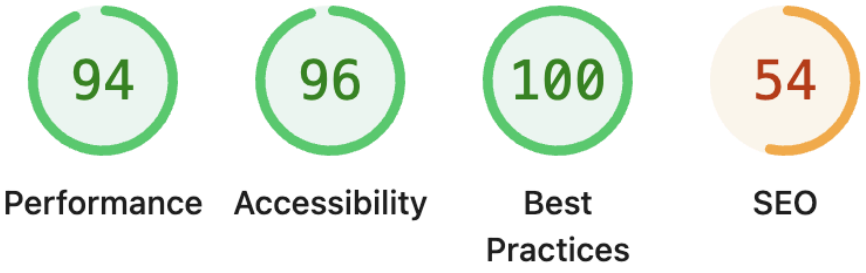
- **Pas de bundle à compiler** : Asset Mapper sert directement les fichiers du dossier assets/ grâce à une importmap, sans étape Webpack. Chaque fichier est servi avec un nom **versionné** (une empreinte du contenu est ajoutée au nom). Le navigateur peut donc garder les fichiers en cache longtemps, et dès qu'un fichier est modifié son nom change : le cache est invalidé automatiquement.
- **Préchargement des modules** : la fonction Twig importmap('app') génère des balises modulepreload, qui permettent au navigateur de télécharger en parallèle les modules JS dont la page aura besoin (Stimulus, Turbo, le contrôleur de l'éditeur).
- **Navigation avec Turbo** : Symfony UX Turbo est activé. Au clic sur un lien ou à l'envoi d'un formulaire, Turbo récupère la nouvelle page en arrière-plan et remplace seulement le <body>. Le CSS et le JS déjà chargés ne sont pas téléchargés à nouveau, et la navigation paraît quasi instantanée.
- **Polices optimisées** : des balises preconnect vers fonts.googleapis.com et fonts.gstatic.com ouvrent la connexion au plus tôt, et le paramètre display=swap affiche le texte immédiatement avec une police système en attendant que Poppins soit chargée. Le texte n'est donc jamais invisible pendant le chargement.
- **Images limitées** : la contrainte Image des formulaires limite les uploads à **2 Mo** et aux formats JPEG, PNG et WebP. Cela évite qu'une image énorme ralentisse le dashboard ou l'éditeur.
- **Requêtes ciblées en base** : findByOwner() ne récupère que les présentations de l'utilisateur connecté, déjà triées par la base (ORDER BY sur la date de création) plutôt qu'en PHP.

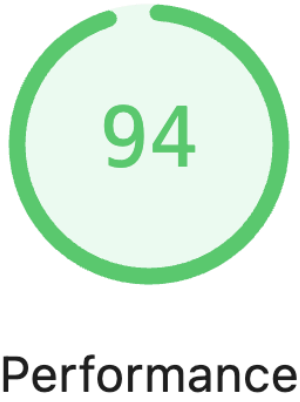
Côté éditeur :

- **Tout le rendu se fait dans le navigateur** : à chaque réglage, le canvas est redessiné localement, sans aucune requête vers le serveur. L'export PNG (canvas.toDataURL) est lui aussi 100 % local : aucun fichier lourd ne transite par le réseau.
- **Aperçu instantané des images** : quand l'utilisateur choisit un fichier, il est affiché tout de suite grâce à URL.createObjectURL(), avant même l'envoi du formulaire. L'ancienne URL temporaire est libérée avec URL.revokeObjectURL(), ce qui évite d'accumuler des images en mémoire.
- **Chargement asynchrone** : les images sont chargées avec new Image() et le canvas n'est redessiné qu'une fois l'image prête (onload), ce qui ne bloque jamais l'interface.

Pistes d'optimisation identifiées :

- ne charger que les graisses de Poppins réellement utilisées : aujourd'hui les 18 variantes (100 à 900, normal et italique) sont demandées à Google Fonts ;
- remplacer la feuille complète de Font Awesome, chargée depuis un CDN, par les seules icônes utilisées (ou par des SVG intégrés) ;
- générer des miniatures redimensionnées pour le dashboard au lieu d'afficher l'image d'origine, et ajouter loading="lazy" pour ne charger les images que lorsqu'elles deviennent visibles ;
- en production, lancer php bin/console asset-map:compile pour que les assets soient servis comme de simples fichiers statiques par le serveur web.





Analyse Desktop

B — CSS

Fichiers séparés pour la maintenance, importés depuis assets/styles/app.css :

Fichier	**Rôle**
`_tokens.css`	couleurs, typo, radius (design tokens)
`_base.css`	reset / styles globaux
`components/_header.css`	header
`components/_buttons.css`	boutons
`components/_forms.css`	champs, dropzone image
`components/_range.css`	sliders

Fichier	Rôle
`components/_cards.css`	cartes dashboard / choix format
`pages/_home.css`	accueil
`pages/_dashboard.css`	liste des présentations
`pages/_editor.css`	layout éditeur (sidebar + canvas)

Changer `--color-primary` dans les tokens met à jour boutons, accents, dropzones.

C — JavaScript

1 — Vision globale

Le JS sert surtout l'**éditeur**. Stimulus relie le DOM (`data-controller="presentation-editor"`) à une classe ES module.

Le contrôleur lit les champs du formulaire Symfony (couleurs, radios, ranges, fichiers) et **redessine le canvas à chaque changement**, sans recharger la page. À l'enregistrement, ce sont les valeurs du formulaire qui partent au serveur : le canvas n'est qu'un aperçu de la recette.

Pourquoi Stimulus ? Stimulus est un petit framework JavaScript (de la famille Hotwire, comme Turbo) intégré à Symfony via le stimulus-bundle. Contrairement à React ou Vue, il ne génère pas le HTML : la page est rendue côté serveur par Twig, et Stimulus vient seulement « brancher » du comportement dessus. Pour Briefed, c'est exactement le besoin : des pages classiques, plus un éditeur interactif.

Un contrôleur Stimulus repose sur trois notions, toutes utilisées dans l'éditeur :

- **Les targets** (static targets) : les éléments HTML dont le contrôleur a besoin, déclarés dans le template avec data-presentation-editor-target="...". L'éditeur en utilise plus d'une trentaine (le canvas, les champs de couleur, les sliders, les champs numériques, les boutons Centrer et Exporter, les aperçus d'image…). En JS, on y accède simplement avec this.canvasTarget, this.scaleInputTarget, etc., sans querySelector.
- **Les values** (static values) : des données passées par Twig au JS via des attributs data-*. Ici bg (couleur de fond), imageUrl, backgroundImageUrl et format.
- **Le cycle de vie** : la méthode connect() est appelée automatiquement dès que l'élément apparaît dans la page, y compris après une navigation Turbo. Pas besoin d'écouter DOMContentLoaded.

Autres scripts générés / présents :

- app.js — point d'entrée de l'importmap : il charge Stimulus et la feuille de style app.css ;
- stimulus_bootstrap.js — démarrage Stimulus ;
- csrf_protection_controller.js — protection CSRF Stimulus (bundle).

2 — `presentation_editor_controller.js` en détail

Au connect() (montage du contrôleur) :

1. Définit les tailles de canvas : paysage 1000×563, standard 1000×750, carré 750×750.
2. Affiche les bons champs de fond (solide / dégradé / image).
3. Recharge l'image source et l'image de fond si elles existent déjà (réédition).
4. Synchronise chaque slider avec un champ numérique.
5. Écoute input / change → appelle draw().

Méthodes importantes :

- **draw()** : peint le fond (fillRect, createLinearGradient, ou drawImage du background), puis l'ombre, la bordure (roundRect + stroke), puis l'image principale clipée (clip + drawImage).
- **center()** : calcule posX / posY pour centrer l'image à l'échelle actuelle.
- **export()** : canvas.toDataURL("image/png") + lien de téléchargement. Rien n'est envoyé au serveur.
- **Changement d'échelle** : ajuste posX / posY pour que le zoom se fasse depuis le centre de l'image, pas depuis le coin supérieur gauche.

Les valeurs initiales du canvas viennent d'attributs data-presentation-editor-*-value remplis par Twig (couleur, URLs d'images, format). C'est le pont backend → front.

Synchroniser un slider et son champ numérique. Chaque réglage (position X/Y, échelle, bordure, opacité, radius, ombre, angle du dégradé) est proposé sous deux formes : un slider pour régler à la souris, et un champ numérique pour saisir une valeur précise. Les deux s'écoutent mutuellement : quand l'un change, l'autre est mis à jour, puis le canvas est redessiné.

```
this.radiusInputTarget.addEventListener("input", (event) => {
   this.radiusNumberInputTarget.value = Number(event.target.value);
   this.draw();
});

this.radiusNumberInputTarget.addEventListener("input", (event) => {
   this.radiusInputTarget.value = Number(event.target.value);
   this.draw();
});
```

Seul le slider porte le nom du champ Symfony : c'est donc sa valeur qui est envoyée au serveur à l'enregistrement.

Le rendu du canvas, calque par calque. draw() repeint toute l'image à chaque appel, dans un ordre précis (comme des calques) :

1. **Le fond** : selon le type choisi, une couleur unie (fillRect), un dégradé linéaire ou une image de fond étirée sur tout le canvas (drawImage). Pour le dégradé, l'angle choisi par l'utilisateur (0 à 360°) est converti en radians, puis Math.cos et Math.sin donnent les points de départ et d'arrivée de part et d'autre du centre du canvas, passés à createLinearGradient.

2. **L'ombre** (drawShadow()) : deux styles sont proposés. « spread » est une ombre large et diffuse (flou 48 px, distance 14 px), « hug » une ombre courte et nette (flou 4 px, distance 8 px). L'angle et l'opacité réglés par l'utilisateur servent à calculer le décalage (shadowOffsetX / shadowOffsetY) et la couleur rgba(0, 0, 0, opacité).
3. **La bordure** : un rectangle arrondi (roundRect) tracé avec stroke(), dont la couleur hexadécimale est convertie en rgba par la méthode hexToRGB() pour appliquer l'opacité choisie.
4. **L'image principale** : ctx.save(), puis un rectangle arrondi utilisé comme **masque** (clip()), drawImage(), et enfin ctx.restore(). C'est ce qui permet d'arrondir les coins de l'image elle-même. save() / restore() évitent que le masque ou l'ombre ne s'appliquent aux dessins suivants.

```
ctx.save();
ctx.beginPath();
ctx.roundRect(x, y, width, height, radius);
ctx.clip();
ctx.drawImage(this.image, x, y, width, height);
ctx.restore();
```

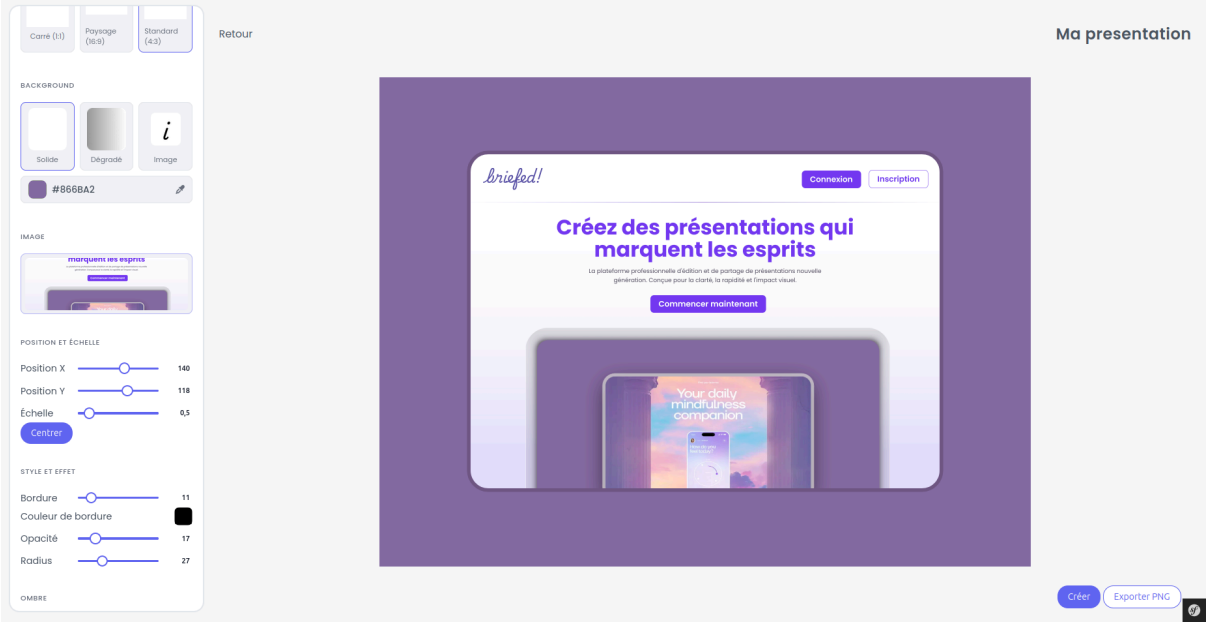
Le chargement des images. Les méthodes loadOverlayImage() et loadBackgroundImage() créent un objet Image et attendent l'événement onload avant de redessiner : le chargement d'une image est asynchrone, et dessiner trop tôt afficherait un canvas vide. Quand l'utilisateur sélectionne un fichier, il est affiché immédiatement grâce à URL.createObjectURL(), sans attendre l'envoi du formulaire. À la réédition, ce sont les URLs des fichiers déjà enregistrés (transmises par Twig) qui sont chargées. Une classe is-filled est ajoutée à la zone de dépôt pour adapter son style une fois l'image choisie.

Le zoom depuis le centre. Si on changeait seulement l'échelle, l'image grandirait vers la droite et vers le bas, car sa position correspond à son coin supérieur gauche. Pour que le zoom reste centré, je calcule la différence de taille entre l'ancienne et la nouvelle échelle, et je décale la position de la moitié de cette différence :

```
const newX = currentX + (previousSize.width - size.width) / 2;
const newY = currentY + (previousSize.height - size.height) / 2;
```

L'affichage conditionnel des champs de fond. updateBackgroundFields() lit le bouton radio coché (solide, dégradé ou image) et affiche seulement les champs utiles grâce à l'attribut hidden. Par exemple, le second sélecteur de couleur et l'angle n'apparaissent qu'en mode dégradé.

L'export PNG. export() vérifie d'abord qu'une image est chargée, puis crée un lien <a> temporaire dont l'attribut download vaut presentation.png et dont l'adresse est le contenu du canvas encodé en PNG (toDataURL). Un click() simulé déclenche le téléchargement.




```
draw() : void  {
    const canvas = this.canvasTarget;
    const ctx = canvas.getContext( contextId: "2d");
    const type : any | string  =
        this.bgTypeInputTargets.find((radio) : boolean  => radio.checked)?.value ??
        "solid";

    if (type === "solid") {
        ctx.fillStyle = this.bgValue || this.color1InputTarget.value;
        ctx.fillRect( x: 0,  y: 0, canvas.width, canvas.height);
    } else if (type === "degrade") {
        const color1 = this.color1InputTarget.value;
        const color2 = this.color2InputTarget.value;
        const angle : number  = Number(this.gradientAngleInputTarget.value);

        const cx : number  = canvas.width / 2;
        const cy : number  = canvas.height / 2;

        const radians : number  = (angle * Math.PI) / 180;
        const length : number  = Math.max(canvas.width, canvas.height);

        const x1 : number  = cx - Math.cos(radians) * length;
        const y1 : number  = cy - Math.sin(radians) * length;
        const x2 : number  = cx + Math.cos(radians) * length;
        const y2 : number  = cy + Math.sin(radians) * length;

        const gradient : CanvasGradient  = ctx.createLinearGradient(x1, y1, x2, y2);
        gradient.addColorStop( offset: 0, color1);
        gradient.addColorStop( offset: 1, color2);
        ctx.fillStyle = gradient;
        ctx.fillRect( x: 0,  y: 0, canvas.width, canvas.height);
    } else if (type === "image" && this.backgroundImage) {
        ctx.drawImage(
            this.backgroundImage,
            0,
            0,
            canvas.width,
            canvas.height,
        );
    }
```

VI — Sécurité

Symfony 7.4 + commandes make:user / make:auth / make:registration-form posent les bases : hachage, sessions, CSRF login.

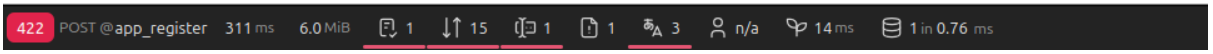
Une application où chacun enregistre ses propres fichiers et ses propres données doit garantir trois choses : qu'un visiteur ne puisse pas se faire passer pour un autre, qu'un utilisateur n'accède qu'à ses propres présentations, et que les données envoyées ne puissent pas être détournées. Voici comment Briefed y répond.

Points concrets dans Briefed :

1. **Mots de passe hachés** (password_hashers: auto), jamais en clair. Le mode auto choisit le meilleur algorithme disponible (bcrypt ou Argon2), qui intègre un sel aléatoire : deux utilisateurs ayant le même mot de passe n'auront pas le même hash, et une fuite de la base ne révélerait pas les mots de passe. À la connexion, Symfony hache le mot de passe saisi et le compare au hash stocké.
2. **CSRF** : login, et suppression (csrf_token('delete' ~ id)). Une attaque CSRF consiste à piéger un utilisateur connecté pour qu'il envoie, sans le savoir, une requête vers Briefed depuis un autre site (par exemple un formulaire caché qui supprimerait ses présentations). Le jeton, unique et lié à la session, est vérifié côté serveur avec isCsrfTokenValid() : sans le bon jeton, la suppression n'a pas lieu. Les formulaires Symfony (création, édition, inscription) embarquent eux aussi automatiquement un jeton CSRF.
3. **Validation serveur** : attributs #[Assert\…] sur les entités (titre, format, couleurs hex #RRGGBB, plages d'échelle / bordure / ombre) + contraintes Image sur les uploads. Le front peut aider l'UX, le serveur décide. Un utilisateur peut modifier le HTML avec les outils du navigateur ou envoyer une requête à la main : seules des valeurs connues sont acceptées (Choice pour le format carre / paysage / standard, le type de fond et le type d'ombre ; Range pour les valeurs numériques ; Regex pour les couleurs ; Email et UniqueEntity pour l'adresse email).
4. **Contrôle d'accès** : #[IsGranted('ROLE_USER')] sur tout /presentations. L'attribut est placé sur la classe du contrôleur : toutes ses routes sont protégées d'un coup, sans risque d'oublier une méthode. Un visiteur non connecté est redirigé vers la page de connexion.
5. **IDOR** : sur edit et delete, comparaison owner / utilisateur courant → 403 sinon. On ne se contente pas de « cacher » le lien dans le dashboard. IDOR (Insecure Direct Object Reference) signifie qu'il suffirait de changer l'identifiant dans l'URL (/presentations/12/edit → /presentations/13/edit) pour accéder à la présentation de quelqu'un d'autre. Le contrôle est fait côté serveur, à chaque requête.
6. **Owner fixé côté serveur** à la création, pas dans le formulaire. Le champ « propriétaire » n'existe pas dans PresentationType : il est impossible de créer une présentation au nom d'un autre utilisateur en ajoutant un champ caché.
7. **Upload** : types MIME autorisés, taille max, nom slugifié + unique, stockage hors Git. La base ne contient que le nom de fichier. L'extension finale est déterminée par guessExtension() à partir du **contenu réel** du fichier, et non du nom fourni par l'utilisateur : un fichier script.php renommé en image.png est rejeté par la contrainte Image, et un fichier accepté ne peut pas recevoir une extension exécutable.

8. **Pas d'admin** dans cette version : pas de back-office, pas de ROLE_ADMIN actif (access_control commenté dans security.yaml).
9. **Protection contre les failles XSS** : Twig échappe automatiquement toutes les variables affichées avec {{ }}. Si un utilisateur donne à sa présentation le titre <script>alert(1)</script>, il est affiché comme du texte et jamais exécuté par le navigateur.
10. **Protection contre les injections SQL** : je n'écris aucune requête SQL à la main. Doctrine génère des **requêtes préparées**, et les valeurs sont toujours passées en paramètres (setParameter('owner', $owner) dans findByOwner), jamais concaténées dans la requête.
11. **Secrets hors du dépôt** : les fichiers .env et .env.local (mot de passe de la base, APP_SECRET) sont exclus par le .gitignore. Le dépôt GitHub étant public, aucun identifiant ne doit s'y trouver.
12. **Sessions et déconnexion** : l'utilisateur connecté est conservé en session par le firewall Symfony (main). La route de déconnexion (app_logout) est interceptée par le firewall, qui invalide la session.





Mentions légales et RGPD. Briefed collecte peu de données personnelles : une adresse email, un mot de passe (stocké haché) et les images que l'utilisateur choisit d'envoyer. C'est le principe de **minimisation** du RGPD : ne collecter que ce qui est nécessaire au service. Le seul cookie utilisé est le cookie de session, strictement nécessaire au fonctionnement de la connexion : il n'a pas besoin de bandeau de consentement. En revanche, la version actuelle n'a pas encore :

- de page de **mentions légales** (identité de l'éditeur du site, hébergeur, contact) ;
- de **politique de confidentialité** expliquant quelles données sont collectées, pourquoi, combien de temps elles sont conservées et comment exercer ses droits ;
- de **droit à l'effacement** : on peut supprimer une présentation, mais pas encore son compte avec l'ensemble de ses données et fichiers (voir partie IX) ;
- d'hébergement local des polices : Poppins est chargée depuis Google Fonts, ce qui transmet l'adresse IP du visiteur à Google. Héberger la police sur le serveur de Briefed éviterait ce transfert.

Ces points seraient indispensables avant une vraie mise en ligne.

HTTPS en production. En local, l'application tourne en HTTP. En production, le HTTPS serait obligatoire : sans chiffrement, le mot de passe envoyé à la connexion et le cookie de session circuleraient en clair sur le réseau et pourraient être interceptés. Un certificat gratuit (Let's Encrypt), proposé par la plupart des hébergeurs, suffit. Il faudrait aussi rediriger toutes les requêtes HTTP vers HTTPS et marquer le cookie de session comme `secure`.

VII — Tests

PHPUnit est installé (phpunit/phpunit en require-dev), le dossier tests/ ne contient pour l'instant que bootstrap.php. **Aucun test automatisé n'est écrit à ce stade.**

J'ai choisi de donner la priorité aux fonctionnalités du MVP dans le temps imparti. L'outillage est pourtant prêt : PHPUnit, symfony/browser-kit et symfony/css-selector (qui permettent de simuler un navigateur dans des tests fonctionnels), le fichier de configuration phpunit.dist.xml, ainsi qu'une configuration dédiée à l'environnement de test dans security.yaml (when@test), qui réduit le coût du hachage des mots de passe pour que les tests s'exécutent plus vite.

Tests manuels. Pendant le développement, j'ai vérifié le fonctionnement dans le navigateur au fur et à mesure :

Scénario	**Résultat attendu**
Accéder à `/presentations` sans être connecté	redirection vers la page de connexion
S'inscrire avec un email déjà utilisé	message « Cet email est déjà utilisé. »
S'inscrire avec un mot de passe de moins de 6 caractères	message d'erreur, compte non créé

Scénario	Résultat attendu
Envoyer une image de plus de 2 Mo ou au mauvais format	message d'erreur, fichier refusé
Modifier l'id dans l'URL d'édition pour viser une autre présentation	erreur 403
Créer, modifier puis supprimer une présentation	la liste du dashboard est mise à jour, le fichier est supprimé
Exporter une présentation	téléchargement d'un fichier `presentation.png`

Ces tests manuels ont une limite : il faut les refaire à chaque modification, et on peut facilement en oublier un.

Ce que des tests automatisés apporteraient. On distingue deux grands types de tests :

- les **tests unitaires**, qui vérifient une petite partie du code isolée (une méthode, un service) ;
- les **tests fonctionnels**, qui simulent un utilisateur : ils envoient une requête HTTP à l'application et vérifient la réponse (code HTTP, redirection, contenu de la page).

Les tests que j'écrirais en priorité :

- test fonctionnel : un anonyme qui tape /presentations est redirigé / refusé ;
- test fonctionnel : un utilisateur A ne peut pas GET /presentations/{id}/edit d'un utilisateur B (403) ;
- test unitaire : FileUploader produit un nom slugifié ;
- test d'inscription : mot de passe trop court rejeté.

Exemple de ce que donnerait le premier test fonctionnel avec WebTestCase :

```
public function testAnonymousIsRedirectedToLogin(): void
{
    $client = static::createClient();
    $client->request('GET', '/presentations');

    $this->assertResponseRedirects('/login');
}
```

Automatiser ces tests permettrait de les relancer en une seule commande (`php bin/phpunit`) après chaque modification, et de s'assurer qu'une nouvelle fonctionnalité ne casse pas la sécurité existante (non-régression). Ils pourraient ensuite être exécutés automatiquement à chaque push avec GitHub Actions.

VIII — Déploiement

Briefed n'est pas encore déployé en ligne. L'application tourne aujourd'hui en **environnement local** : le serveur Symfony pour PHP, et Docker Compose pour la base

(MariaDB 10.11, port 3308) et phpMyAdmin (port 8080). J'ai préféré consacrer le temps du projet à un MVP stable et sécurisé plutôt qu'à une mise en ligne précipitée. Voici la démarche que je suivrais pour le mettre en production.

Pistes techniques adaptées à **ce** projet (Asset Mapper, pas Webpack) :

1. **Hébergeur PHP 8.2+ / MariaDB ou MySQL** (o2switch, OVH, etc.). Un hébergement mutualisé suffit pour une application de cette taille. Un VPS donnerait plus de liberté (réutiliser Docker, par exemple) mais demanderait plus de maintenance.
2. **Nom de domaine**, relié à l'hébergement par un enregistrement DNS.
3. **Transfert des fichiers** (Git pull ou FTP) ; la racine web = dossier public/. C'est essentiel : seul public/index.php (et les assets) doit être accessible depuis le web. Le code source, config/, .env et vendor/ restent hors de portée des visiteurs.
4. **`composer install --no-dev --optimize-autoloader`** : installe uniquement les dépendances de production (sans le Maker Bundle, le Web Profiler ou PHPUnit) et optimise le chargement des classes.
5. **`.env` de production** (APP_SECRET, DATABASE_URL) — **jamais** commité avec des secrets. On y passe aussi `APP_ENV=prod` et `APP_DEBUG=0` : en production, les erreurs détaillées et la barre de debug ne doivent jamais être affichées, car elles révéleraient des informations sur le code.
6. **Migrations** : `php bin/console doctrine:migrations:migrate --no-interaction`. Les fichiers de migration versionnés dans `migrations/` recréent exactement le même schéma qu'en local.
7. **Droits d'écriture** sur `public/uploads/` (images envoyées par les utilisateurs) et `var/` (cache et logs).
8. **HTTPS (certificat SSL)**, par exemple avec Let's Encrypt, et redirection automatique de HTTP vers HTTPS (voir partie VI).
9. **Pas de `npm run build`** : Asset Mapper n'a pas besoin de compilation Webpack. En prod : `php bin/console asset-map:compile`, qui copie les assets versionnés dans `public/assets/` pour qu'ils soient servis directement par le serveur web.

Il faudrait aussi vider et préchauffer le cache de production (php bin/console cache:clear --env=prod) et configurer le serveur web pour que toutes les requêtes passent par public/index.php : un fichier .htaccess (fourni par symfony/apache-pack) pour Apache, ou un bloc de configuration équivalent pour Nginx.

Vérifications après la mise en ligne : inscription et connexion, création d'une présentation avec upload d'image, affichage des images dans le dashboard, contrôle d'accès (403 sur la présentation d'un autre utilisateur), export PNG, redirection HTTPS, et un audit Lighthouse sur la version en ligne.

Mises à jour : pour publier une nouvelle version, il suffirait de répéter une courte séquence : git pull, composer install --no-dev --optimize-autoloader, migrations, asset-map:compile puis vidage du cache. À terme, cette séquence pourrait être automatisée (script de déploiement ou GitHub Actions).

IX — Améliorations et perspectives d'évolution

A — Améliorations (proches du code actuel)

1 — Confirmation d'email et mot de passe oublié

L'inscription ne vérifie pas l'email. Un lien avec token (bundle Symfony) activerait le compte. Même principe pour « mot de passe oublié ».

Aujourd'hui, n'importe qui peut créer un compte avec une adresse qui ne lui appartient pas, et un utilisateur qui oublie son mot de passe n'a aucun moyen de récupérer son compte. Symfony propose deux bundles adaptés : symfonycasts/verify-email-bundle et symfonycasts/reset-password-bundle. La commande make:registration-form peut d'ailleurs générer la vérification d'email. Le principe : après l'inscription, un email contenant un lien signé et limité dans le temps est envoyé ; tant que le lien n'a pas été cliqué, un champ isVerified reste à false et le compte n'est pas actif. Le composant symfony/mailer, nécessaire à l'envoi des emails, est déjà installé dans le projet. Pour le mot de passe oublié, le jeton est stocké haché en base et expire au bout d'un court délai.

2 — Durcissement des mots de passe

Passer de 6 caractères à une politique plus stricte (longueur, complexité), alignée RGPD / ANSSI.

Concrètement, Symfony fournit des contraintes prêtes à l'emploi : PasswordStrength, qui évalue la robustesse du mot de passe, et NotCompromisedPassword, qui vérifie que le mot de passe n'apparaît pas dans une fuite de données connue (base Have I Been Pwned). La CNIL recommande par exemple au moins 12 caractères mêlant majuscules, minuscules, chiffres et caractères spéciaux. En complément, l'option login_throttling du firewall limiterait le nombre de tentatives de connexion et freinerait les attaques par force brute.

3 — Suppression du compte (RGPD)

Aujourd'hui on peut supprimer une présentation, pas le compte ni les fichiers associés en cascade. Il faudrait une action « supprimer mon compte » qui retire User + présentations + fichiers.

Cette action serait protégée par un jeton CSRF et une confirmation, comme la suppression d'une présentation. Côté code, il faudrait parcourir les présentations de l'utilisateur pour supprimer leurs images du disque avec FileUploader::remove(), puis supprimer les entrées en base (avec une option cascade: ['remove'] ou orphanRemoval sur la relation User → Presentation), et enfin déconnecter l'utilisateur en invalidant sa session. On pourrait y ajouter un export de ses données (droit à la portabilité).

4 — Drag & drop sur le canvas

Position et échelle passent par des sliders. Un glisser-déposer (événements souris / pointer sur le canvas) rendrait l'éditeur plus naturel, tout en continuant d'écrire posX / posY dans les champs du formulaire.

J'utiliserais les **Pointer Events** (pointerdown, pointermove, pointerup), qui fonctionnent à la fois à la souris et au doigt sur mobile. Il faudrait convertir les coordonnées du pointeur en coordonnées du canvas, car le canvas est affiché plus petit que sa taille réelle : on utilise getBoundingClientRect() et le rapport entre canvas.width et la largeur affichée. Il faudrait aussi vérifier que le clic tombe bien sur l'image, puis mettre à jour les sliders et les champs numériques avant d'appeler draw(). La molette de la souris pourrait de la même façon piloter le zoom, en réutilisant le calcul de zoom centré qui existe déjà.

5 — Miniature du dashboard

Le dashboard affiche l'**image source**, pas le rendu final (fond + cadre). Générer une miniature au save, ou un aperçu canvas en liste, collerait mieux au produit.

Deux approches sont possibles. Côté navigateur : au moment de l'enregistrement, le contrôleur Stimulus génère une version réduite du canvas avec canvas.toBlob() et l'ajoute au formulaire comme un fichier supplémentaire. Côté serveur : une bibliothèque de traitement d'image (GD ou Imagine) produit la miniature. Dans les deux cas, le dashboard afficherait une image légère, fidèle au rendu final, ce qui améliorerait aussi les performances (voir partie V).

B — Perspectives d'évolution

1 — Galerie / partage public

Lien de partage en lecture seule, ou galerie publique, sans exposer l'éditeur ni permettre l'IDOR.

Pour le lien de partage, chaque présentation recevrait un jeton aléatoire et impossible à deviner (par exemple généré avec bin2hex(random_bytes(16))) et une route publique du type /p/{token}, qui afficherait uniquement le rendu final. On n'utiliserait surtout pas l'identifiant numérique, qui se devine facilement. Un booléen isPublic permettrait à l'utilisateur de choisir ce qu'il partage ou non dans la galerie. Les routes d'édition resteraient protégées par le contrôle de propriété actuel.

2 — Templates de fonds

Bibliothèque de dégradés / images de fond prédéfinis (nouvelle table ou fixtures), pour aller plus vite qu'un color picker vide.

L'entité Background existe déjà et contient toutes les informations nécessaires (type, couleurs, angle, image) : les modèles pourraient être des enregistrements prédéfinis, chargés avec des fixtures (doctrine/doctrine-fixtures-bundle). Dans l'éditeur, ils apparaîtraient sous forme de cartes cliquables ; au clic, le contrôleur Stimulus remplirait les champs du formulaire avec les valeurs du modèle, puis appellerait draw(). L'utilisateur pourrait ensuite retoucher le résultat comme d'habitude.

3 — Plusieurs calques / mockups device

Encadrer la capture dans un faux iPhone / navigateur (comme shots.so), en plus du cadre actuel (radius + ombre).

Techniquement, un mockup est une image de cadre (un téléphone, une fenêtre de navigateur) dessinée sur le canvas autour de l'image de l'utilisateur. Pour gérer plusieurs images ou éléments, il faudrait une nouvelle entité (par exemple Layer), liée à Presentation en OneToMany, avec une position, une taille et un ordre d'affichage. draw() dessinerait alors les calques un par un, dans l'ordre. À ce niveau de complexité, une bibliothèque spécialisée comme Konva ou Fabric.js pourrait devenir pertinente, alors que l'API Canvas native suffit largement pour la version actuelle.

X — Conclusion

Briefed est parti d'un besoin que j'avais moi-même, pouvoir présenter proprement une capture d'écran ou un visuel sans passer par un outil de retouche. Le produit final répond à ce besoin avec un éditeur complet : fond uni, dégradé ou image, cadre, coins arrondis, ombre, position et échelle et un export PNG immédiat.

Le choix technique dont je suis le plus satisfait est d'avoir enregistré en base une **recette** plutôt qu'une image aplatie. Une présentation, c'est un ensemble de réglages liés à un fichier source. Cela rend chaque création rééditable indéfiniment, garde la base légère puisque seuls les noms de fichiers y sont stockés, et permet de faire évoluer le rendu sans toucher aux données existantes.

Ce projet m'a surtout appris à **répartir les responsabilités entre le serveur et le navigateur**. Le serveur décide de tout ce qui engage la sécurité et les données : le propriétaire d'une présentation, la validation des valeurs, le traitement des fichiers envoyés. Le navigateur, lui, se charge de ce qui doit être instantané : l'aperçu du canvas et l'export. C'est la ligne directrice que j'ai suivie à chaque fois que je devais choisir où écrire une fonctionnalité.

J'ai aussi appris à faire des choix et à les assumer. Partir d'un MVP, écarter les fonctionnalités secondaires, et identifier honnêtement ce qui reste à faire : les tests automatisés, la validation d'email, le déploiement, et une expérience d'édition plus naturelle au glisser-déposer. Ces pistes sont détaillées dans la partie précédente, et je compte en traiter plusieurs après la formation.

Cette formation chez IPME m'a fait passer d'une idée de produit sans aucune compétence technique à la capacité de concevoir et de développer une application complète, de la base de données jusqu'à l'interface. Briefed en est la preuve la plus concrète, j'y ai appliqué ce que j'ai appris sur Symfony, Doctrine et la sécurité, en m'appuyant sur ce que je savais déjà du design et des besoins des utilisateurs.

À l'issue de ce titre, j'ai trouvé une alternance de développeur full stack dans une agence digitale, dans le cadre d'une CDA. En parallèle, je compte continuer à faire évoluer Briefed, qui reste le premier outil d'une suite que j'aimerais construire pour les créatifs.